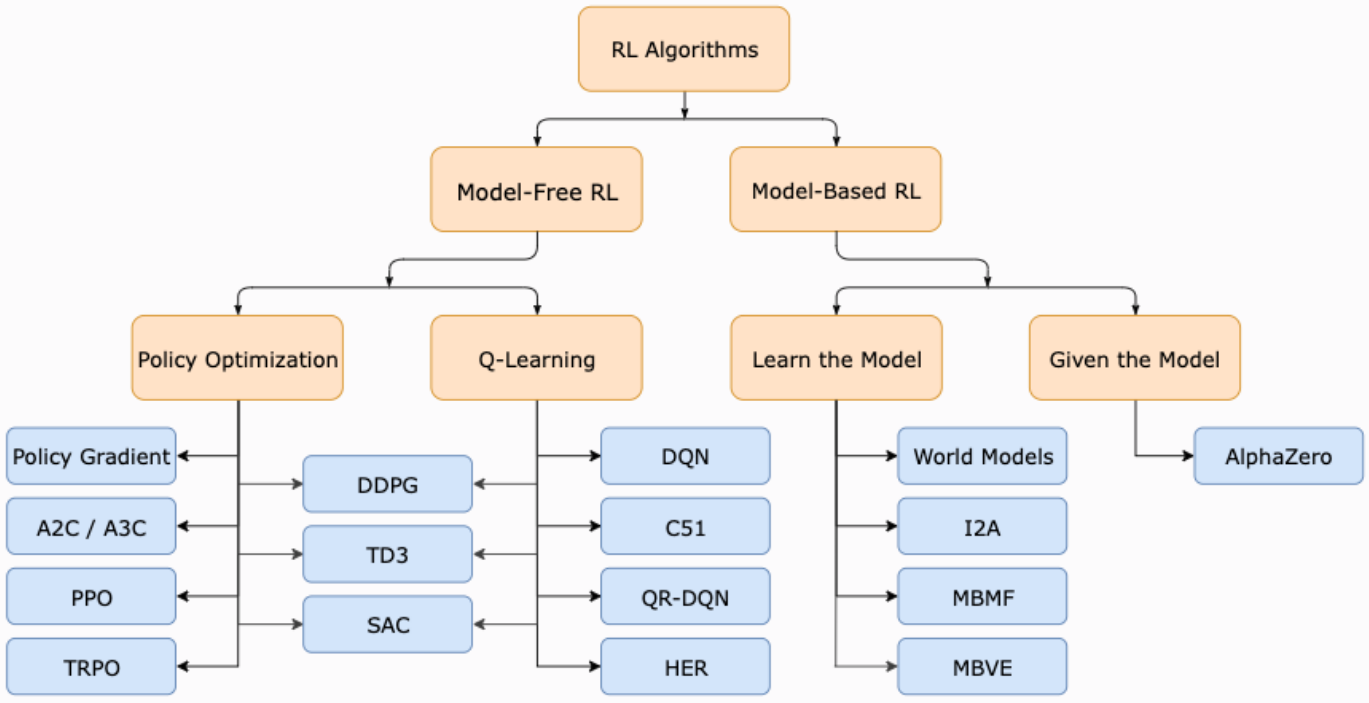
16-831: Introduction to Robot Learning, Fall 2024

Advanced RL Algorithms



Taxonomy of RL algorithms, OpenAI

Instructor: Guanya Shi

LeCAR
Learning and Control for Agile Robotics Lab

Carnegie Mellon University
The Robotics Institute

Recording Reminder

- This session will be audio recorded for educational use by other students in this course.

Course Structure

Latest schedule: https://16-831.github.io/fall24/lectures

- ❑ Topic group 1: Course introduction: What is robot learning and why?
- ❑ Topic group 2: Machine/Deep learning refresher
- ❑ Topic group 3: Imitation learning
- ❑ Topic group 4: Model-free reinforcement learning
- ❑ Topic group 5: Model-based reinforcement learning
- ❑ Topic group 6: Bandit, preference-based learning, exploration
- ❑ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ❑ Topic group 8: Specialized topics (one guest lecture)
- ❑ Project presentation

Outline

- ❑ Taxonomy of MFRL algorithms
  - Recap: Q-learning
  - Recap: Policy gradient
  - Recap: Actor-critic
  - Trade-off between different algorithms
- ❑ Advanced RL Algorithms:
  - NPG: Natural policy gradient
  - TRPO: Trust region policy optimization
  - PPO: Proximal policy optimization
  - Algorithms we have introduced: DQN and extensions, DDPG, SAC

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

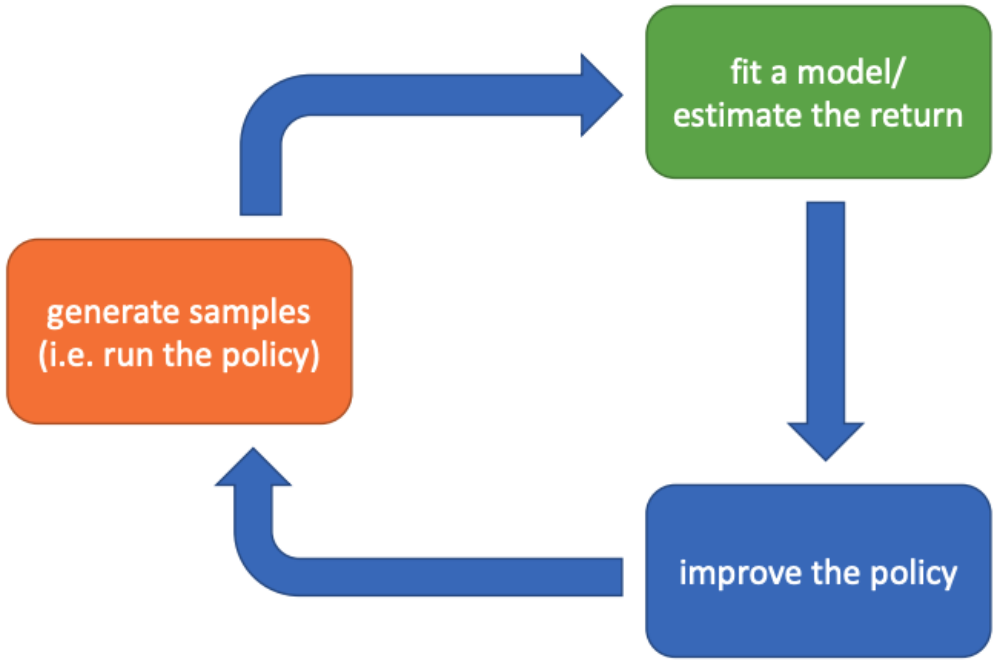
Outline

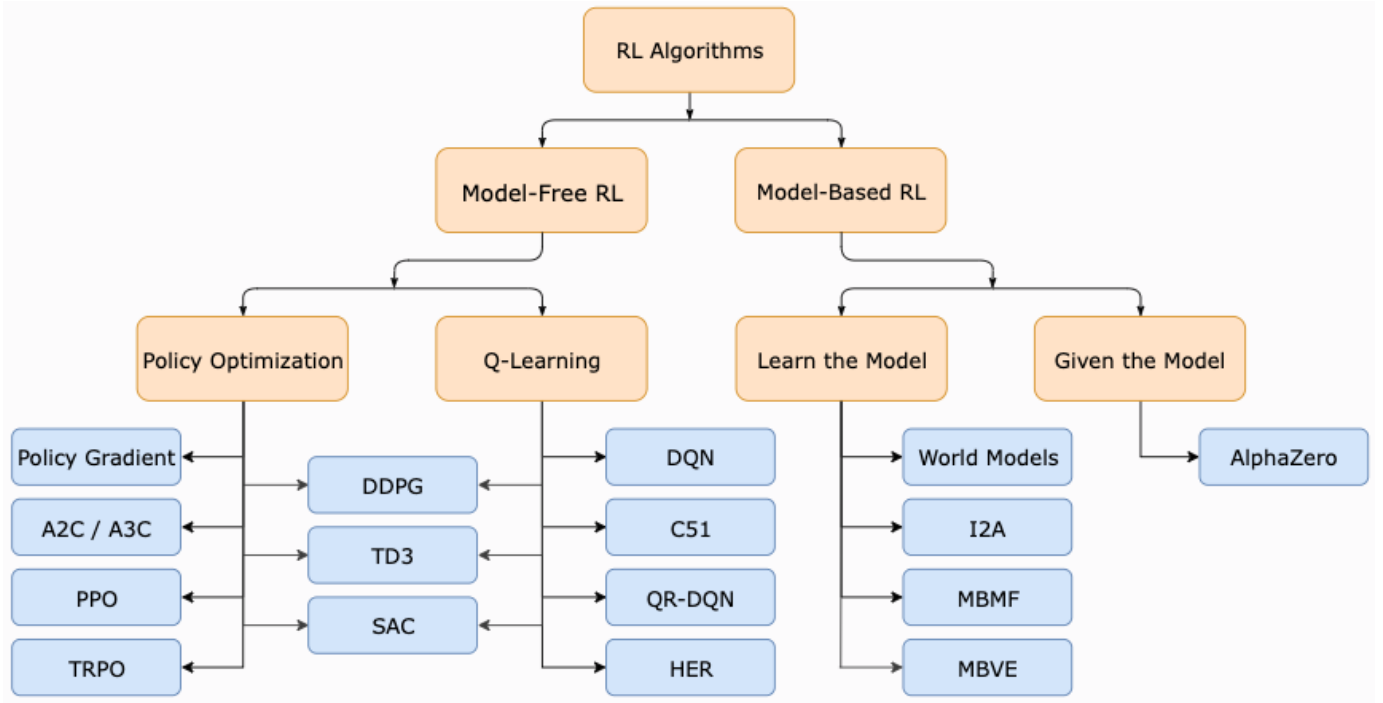
- ❑ Taxonomy of MFRL algorithms
  - Recap: Q-learning
  - Recap: Policy gradient
  - Recap: Actor-critic
  - Trade-off between different algorithms
- ❑ Advanced RL Algorithms:
  - NPG: Natural policy gradient
  - TRPO: Trust region policy optimization
  - PPO: Proximal policy optimization
  - Algorithms we have introduced: DQN and extensions, DDPG, SAC

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Taxonomy of MFRL Algorithms

- **Value-based** (example: FQI, DQN): estimate value function or Q-function of the optimal policy (no explicit policy).
- **Policy-gradient** (example: REINFORCE, NPG): directly optimize the policy
- **Actor-critic** (example: SAC, DDPG): estimate value function or Q-function of the current policy, use it to improve policy





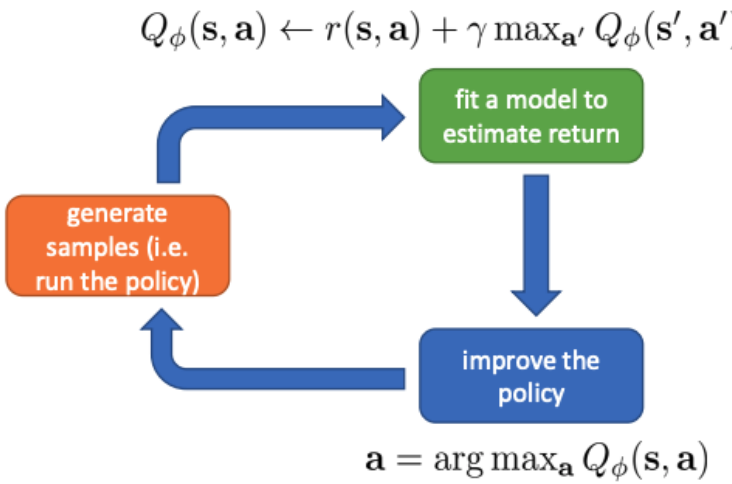
Recap: Q-Learning

- Based on: **Bellman optimality equation**

$$Q^*(s,a) = \sum_{s',r} p(s',r|s,a)[r + \gamma \cdot \max_{a'} Q^*(s',a')], \forall s,a$$

- Example: DQN (Deep Q-Network):
  - Epsilon greedy policy gives a and observe (s, a, s', r)
  - Update the replay buffer
  - Sample a batch from the buffer. Compute the target $y_i = r(s_i, a_i) + \gamma \max_{a_i'} Q_{\phi^-}(s_i', a_i')$
  - Update: $\phi \leftarrow \phi - \eta \sum_i \left(Q_\phi(s_i, a_i) - y_i\right) \nabla_\phi Q_\phi(s_i, a_i)$
  - Periodically update ϕ^-: $\phi^- \leftarrow \phi$

- Extensions: DDQN, prioritized replay buffer, dueling architecture, Q learning with continuous action space, etc



Recap: Policy Gradient

$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^\star = \arg\max_\theta \underbrace{E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

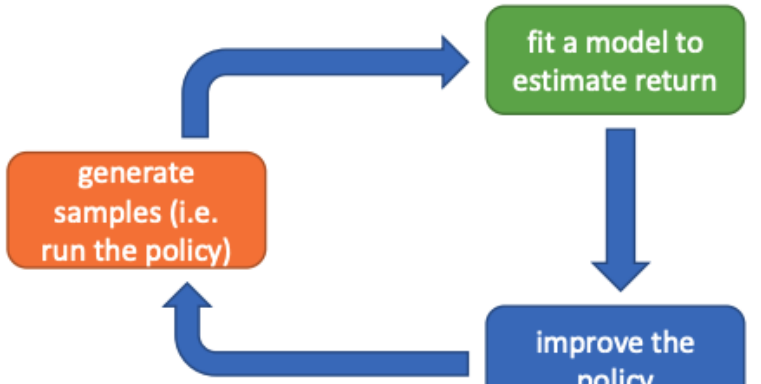
- Based on: **Policy gradient theorem**

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta(\tau)}[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

- Example: REINFORCE

1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

$$\hat{Q}^\pi(\mathbf{x}_t, \mathbf{u}_t) = \sum_{t'=t}^{T} r(\mathbf{x}_{t'}, \mathbf{u}_{t'})$$



$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Recap: Actor-Critic

❑ Based on: **Policy gradient with baselines**

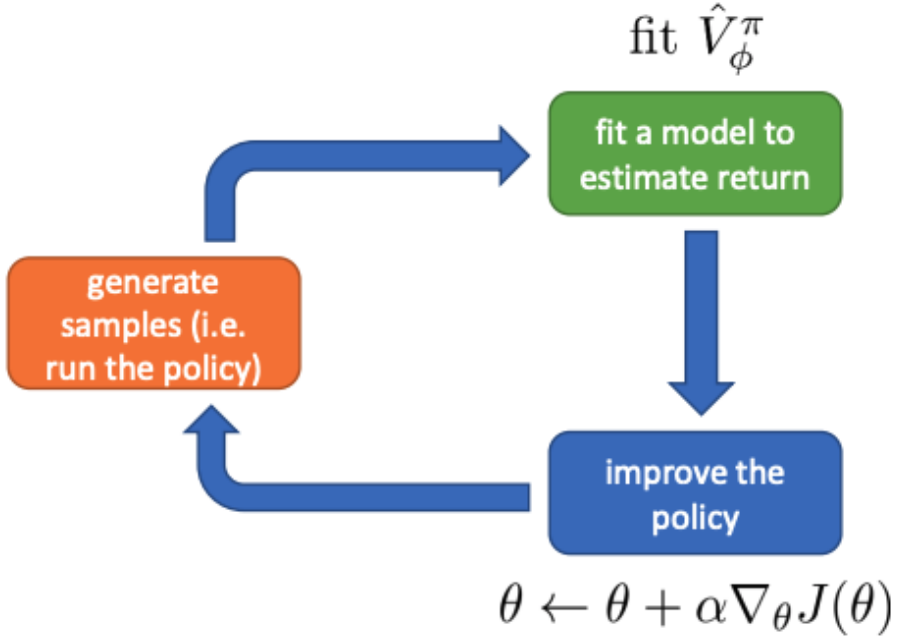
$$\nabla_\theta J(\theta) \propto E_{s\sim p_\theta, a\sim\pi_\theta}[(Q^\pi(s,a) - V^\pi(s))\nabla_\theta \log \pi_\theta(a|s)]$$

- Learn $V_\phi^\pi(s)$
- Estimate $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ by $r + \gamma V_\phi^\pi(s) - V_\phi^\pi(s)$

❑ Example: online actor-critic

online actor-critic algorithm:

1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
2. update $\hat{V}_\phi^\pi$ using target $r + \gamma \hat{V}_\phi^\pi(\mathbf{s}')$
3. evaluate $\hat{A}^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}') - \hat{V}_\phi^\pi(\mathbf{s})$
4. $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}) \hat{A}^\pi(\mathbf{s}, \mathbf{a})$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$



Recap: Unify PG and Actor-Critic

- General advantage estimation (GAE, Schulman et al., 2016):

Policy gradient methods maximize the expected total reward by repeatedly estimating the gradient $g := \nabla_\theta \mathbb{E}\left[\sum_{t=0}^{\infty} r_t\right]$. There are several different related expressions for the policy gradient, which have the form

$$g = \mathbb{E}\left[\sum_{t=0}^{\infty} \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right], \tag{1}$$

where Ψ_t may be one of the following:

1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory.
2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t.
3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula.
4. $Q^\pi(s_t, a_t)$: state-action value function.
5. $A^\pi(s_t, a_t)$: advantage function.
6. $r_t + V^\pi(s_{t+1}) - V^\pi(s_t)$: TD residual.

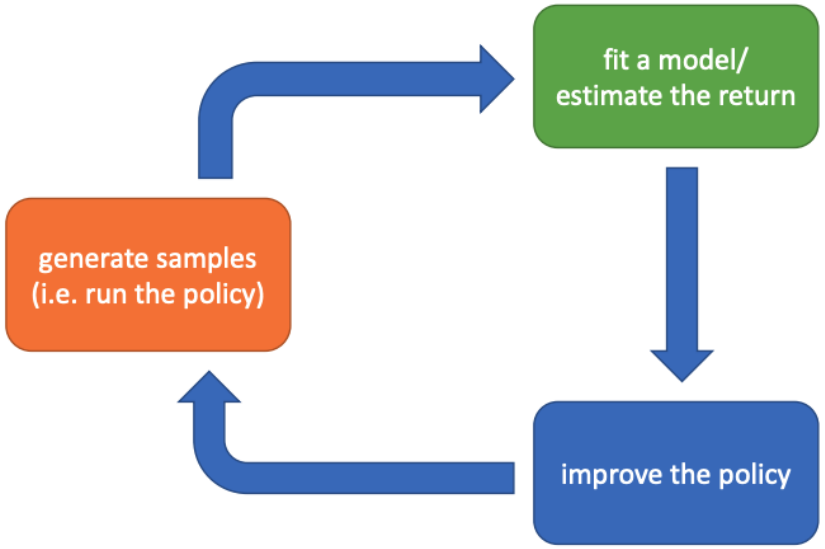
The latter formulas use the definitions

$$V^\pi(s_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t:\infty}}}\left[\sum_{l=0}^{\infty} r_{t+l}\right] \qquad Q^\pi(s_t, a_t) := \mathbb{E}_{\substack{s_{t+1:\infty},\\ a_{t+1:\infty}}}\left[\sum_{l=0}^{\infty} r_{t+l}\right] \tag{2}$$

$$A^\pi(s_t, a_t) := Q^\pi(s_t, a_t) - V^\pi(s_t), \quad \text{(Advantage function)}. \tag{3}$$

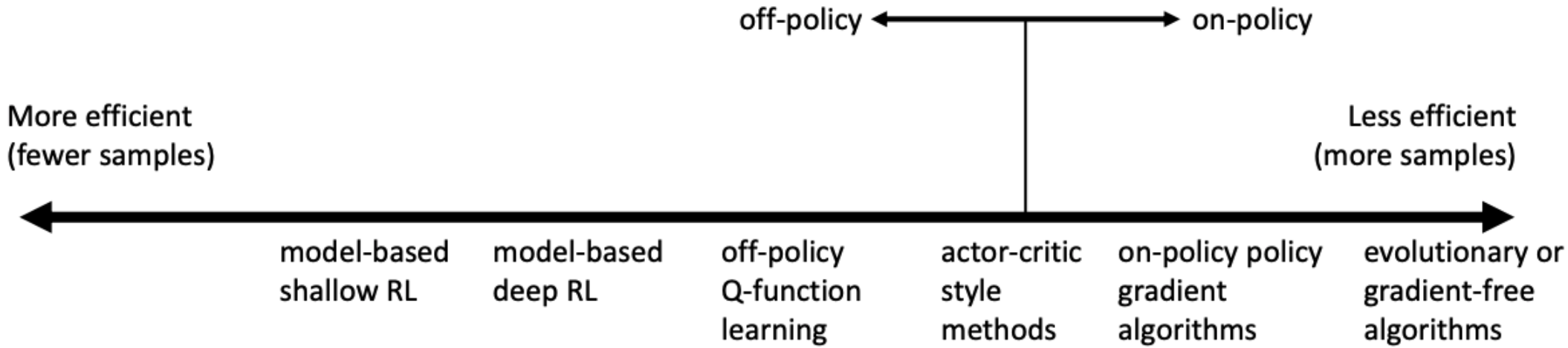
Trade-off Between Algorithms

- ❑ Why so many RL algorithms (even not considering model-based RL)?
- ❑ Different tradeoffs! There is no “perfect” RL algorithm – depending on your applications
  - Sampling efficiency
  - Stability & ease of use
- ❑ Different assumptions
  - Stochastic / deterministic?
  - Continuous / discrete?
  - Finite horizon or infinite horizon?
- ❑ Different things are easy or hard in different settings
  - Learning value/policy/model: which one is easier?



Trade-off Between Algorithms

- Example: sampling efficiency



- This is only a general trend
- Warning: "more efficient" doesn't necessarily mean shorter wall clock time!
  - I.e., generating samples might be super fast (e.g., in simulation)

Outline

- ❑ Taxonomy of MFRL algorithms
  - Recap: Q-learning
  - Recap: Policy gradient
  - Recap: Actor-critic
  - Trade-off between different algorithms
- ❑ Advanced RL Algorithms:
  - NPG: Natural policy gradient
  - TRPO: Trust region policy optimization
  - PPO: Proximal policy optimization
  - Algorithms we have introduced: DQN and extensions, DDPG, SAC

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

NPG (Natural Policy Gradient)

A Natural Policy Gradient

Sham Kakade
Gatsby Computational Neuroscience Unit
17 Queen Square, London, UK WC1N 3AR
http://www.gatsby.ucl.ac.uk
sham@gatsby.ucl.ac.uk

- Recap: policy gradient
  - Rollout π_θ. Compute $\nabla_\theta J(\theta)$
  - Gradient ascent: $\theta' = \theta + \alpha \nabla_\theta J(\theta)$. Get a new policy $\pi_{\theta'}$
- α controls the learning rate. Reformulate it to a constrained optimization problem:

$$\max_{\theta'} (\theta' - \theta)^\top \nabla_\theta J(\theta)$$
$$\text{s.t. } \|\theta - \theta'\|^2 \leq \epsilon$$

- ϵ controls the learning rate.

NPG (Natural Policy Gradient)

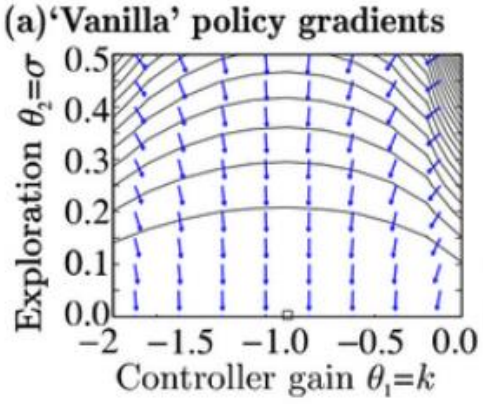
- Recap: policy gradient
  - Rollout π_θ. Compute $\nabla_\theta J(\theta)$
  - Gradient ascent: $\theta' = \theta + \alpha \nabla_\theta J(\theta)$. Get a new policy $\pi_{\theta'}$
- α controls the learning rate. Reformulate it to a constrained optimization problem:

$$\max_{\theta'} (\theta' - \theta)^\top \nabla_\theta J(\theta)$$
$$\text{s.t. } \|\theta - \theta'\|^2 \leq \epsilon$$

- Problem: why is the constraint using 2-norm $\|\theta - \theta'\|^2$?
  - Some part of θ might change π_θ a lot more than others!
- Example:

$$r(\mathbf{s}_t, \mathbf{a}_t) = -\mathbf{s}_t^2 - \mathbf{a}_t^2$$

$$\log \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) = -\frac{1}{2\sigma^2}(k\mathbf{s}_t - \mathbf{a}_t)^2 + \text{const} \qquad \theta = (k, \sigma)$$



(image from Peters & Schaal 2008)

NPG (Natural Policy Gradient)

- Idea: Directly using some distance between two policies

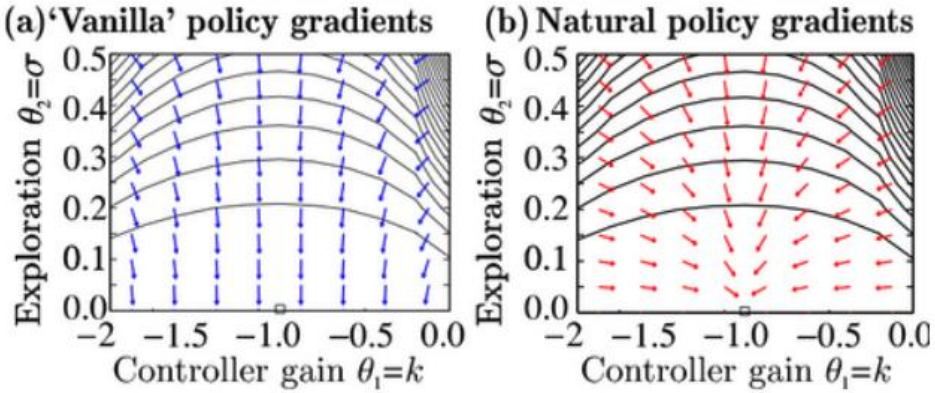
$$\max_{\theta'}(\theta' - \theta)^\top \nabla_\theta J(\theta)$$
$$\text{s.t. } D(\pi_\theta, \pi_{\theta'}) \leq \epsilon$$

- Natural policy gradient: using Fisher-information matrix

$$\max_{\theta'}(\theta' - \theta)^\top \nabla_\theta J(\theta)$$
$$\text{s.t. } (\theta' - \theta)^\top \mathbf{F}(\theta' - \theta) \leq \epsilon$$

$$\mathbf{F} = E_{\pi_\theta}[\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}) \nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s})^T]$$

- The the update law becomes $\theta' = \theta + \alpha \mathbf{F}^{-1} \nabla_\theta J(\theta)$
- Can be viewed as a second-order version of PG



(figure from Peters & Schaal 2008)

TRPO/PPO: Policy Gradient as Policy Iteration

- Recap: policy gradient with value function as baselines

$$J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\sum_t \gamma^t r(s_t, a_t) \right] \qquad \nabla_\theta J(\theta) \propto E_{s \sim p_\theta, a \sim \pi_\theta} [A^{\pi_\theta}(s,a) \nabla_\theta \log \pi_\theta(a|s)]$$

- A policy iteration perspective: $J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)}[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t)]$
- Proof:

$$\begin{aligned}
J(\theta') - J(\theta) &= J(\theta') - E_{\mathbf{s}_0 \sim p(\mathbf{s}_0)}\left[V^{\pi_\theta}(\mathbf{s}_0)\right] \\
&= J(\theta') - E_{\tau \sim p_{\theta'}(\tau)}\left[V^{\pi_\theta}(\mathbf{s}_0)\right] \\
&= J(\theta') - E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t V^{\pi_\theta}(\mathbf{s}_t) - \sum_{t=1}^{\infty} \gamma^t V^{\pi_\theta}(\mathbf{s}_t)\right] \\
&= J(\theta') + E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t))\right] \\
&= E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t)\right] + E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t))\right] \\
&= E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t (r(\mathbf{s}_t, \mathbf{a}_t) + \gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t))\right] \\
&= E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right]
\end{aligned}$$

- Why it makes sense? Sample $\tau = \{s_0, a_0, \cdots\}$ using θ' and evaluate $A^{\pi_\theta}(s,a)$ on it. $A^{\pi_\theta}(s,a)$ is positive -> θ' is better than θ. Otherwise, θ is better than θ'

TRPO/PPO: Policy Gradient as Policy Iteration

- Recap: policy gradient with value function as baselines

$$J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\sum_t \gamma^t r(s_t, a_t) \right] \qquad \nabla_\theta J(\theta) \propto E_{s \sim p_\theta, a \sim \pi_\theta} [A^{\pi_\theta}(s,a) \nabla_\theta \log \pi_\theta(a|s)]$$

- A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t) \right]$$

- Why it could be useful? We can directly do $\max_{\theta'} J(\theta') - J(\theta)$ to improve the policy!
- Why this form is not that useful? Chicken and egg problem! We cannot compute $E_{\tau \sim p_{\theta'}(\tau)}[\cdot]$ without knowing θ'
- **The key idea of TRPO/PPO**: control the distance between θ' and θ such that we can use $E_{\tau \sim p_\theta(\tau)}[\cdot]$ to estimate $J(\theta') - J(\theta)$!

TRPO/PPO: Policy Gradient as Policy Iteration

- A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t) \right]$$

- Using importance sampling again:

$$E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] = \sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)} \left[\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

$$= \sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

- Can we use $s_t \sim p_\theta(s_t)$ to approximate?
- Yes! If $D(\pi_\theta, \pi_{\theta'})$ is small.

TRPO: Trust Region Policy Optimization

- A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)}\left[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t)\right]$$

- TRPO: If $D(\pi_\theta, \pi_{\theta'})$ is small, we have:

$$\sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)}\left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right]\right] \approx \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)}\left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right]\right]$$

- Therefore:

$$\theta' \leftarrow \arg\max_{\theta'} \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)}\left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)\right]\right]$$

$$\text{such that } D_{\mathrm{KL}}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \| \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)) \leq \epsilon$$

Trust Region Policy Optimization

John Schulman JOSCHU@EECS.BERKELEY.EDU
Sergey Levine SLEVINE@EECS.BERKELEY.EDU
Philipp Moritz PCMORITZ@EECS.BERKELEY.EDU
Michael Jordan JORDAN@CS.BERKELEY.EDU
Pieter Abbeel PABBEEL@CS.BERKELEY.EDU
University of California, Berkeley, Department of Electrical Engineering and Computer Sciences

PPO: Proximal Policy Optimization

- A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t) \right]$$

- The objective in TRPO:

$$\theta' \leftarrow \arg\max_{\theta'} \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

$$\text{such that } D_{\mathrm{KL}}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \| \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)) \leq \epsilon$$

- It is a constrained non-convex optimization problem!
  - Complicated and hard/slow to solve
- PPO: design a surrogate objective function

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

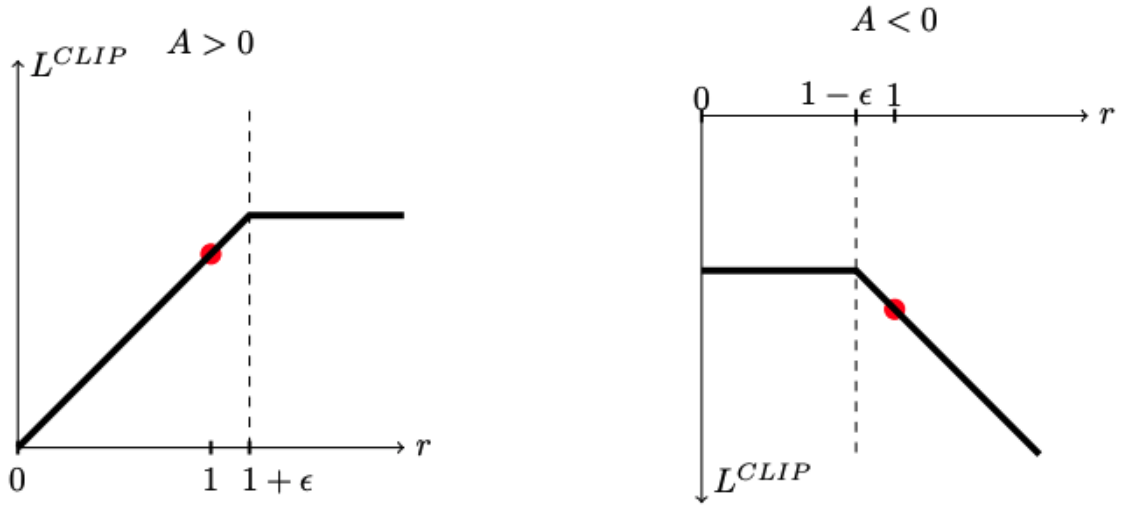
PPO: Proximal Policy Optimization

- The objective function in TRPO:

$$\theta' \leftarrow \arg\max_{\theta'} \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

- PPO: design a surrogate objective function.
- Define $r_t(\theta') = \frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}$. Change the term in the red circle to ($\epsilon = 0.2$ works well):

$$L_t^{CLIP}(\theta') = \min(r_t(\theta')\gamma^t A^{\pi_\theta}(s_t, a_t), \text{clip}(r_t(\theta'), 1-\epsilon, 1+\epsilon)\gamma^t A^{\pi_\theta}(s_t, a_t))$$

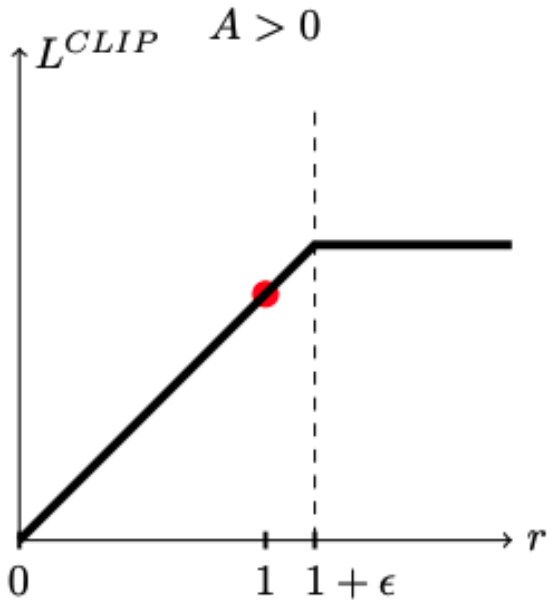


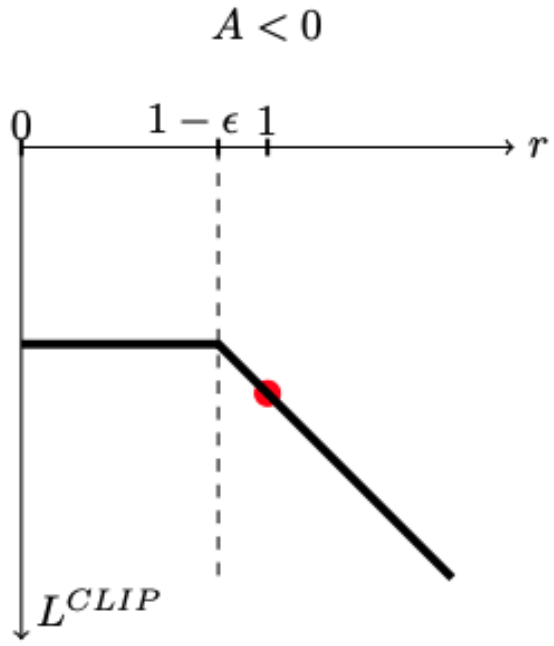
PPO: Proximal Policy Optimization

- Define $r_t(\theta') = \frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}$. Change the term in the red circle to ($\delta = 0.2$ works well):

$$L_t^{CLIP}(\theta') = \min(r_t(\theta')\gamma^t A^{\pi_\theta}(s_t, a_t), \text{clip}(r_t(\theta'), 1-\delta, 1+\delta)\gamma^t A^{\pi_\theta}(s_t, a_t))$$

- Note that $L_t^{CLIP}(\theta') \leq r_t(\theta')\gamma^t A^{\pi_\theta}(s_t, a_t)$. I.e., a pessimistic lower bound!
- If we only have the clip part ($\text{clip}(r_t(\theta'), 1-\delta, 1+\delta)\gamma^t A^{\pi_\theta}(s_t, a_t)$):
  - "Very bad" $r_t(\theta')$ will NOT be punished (e.g., $r_t(\theta') \ll 1$ when $A > 0$)!



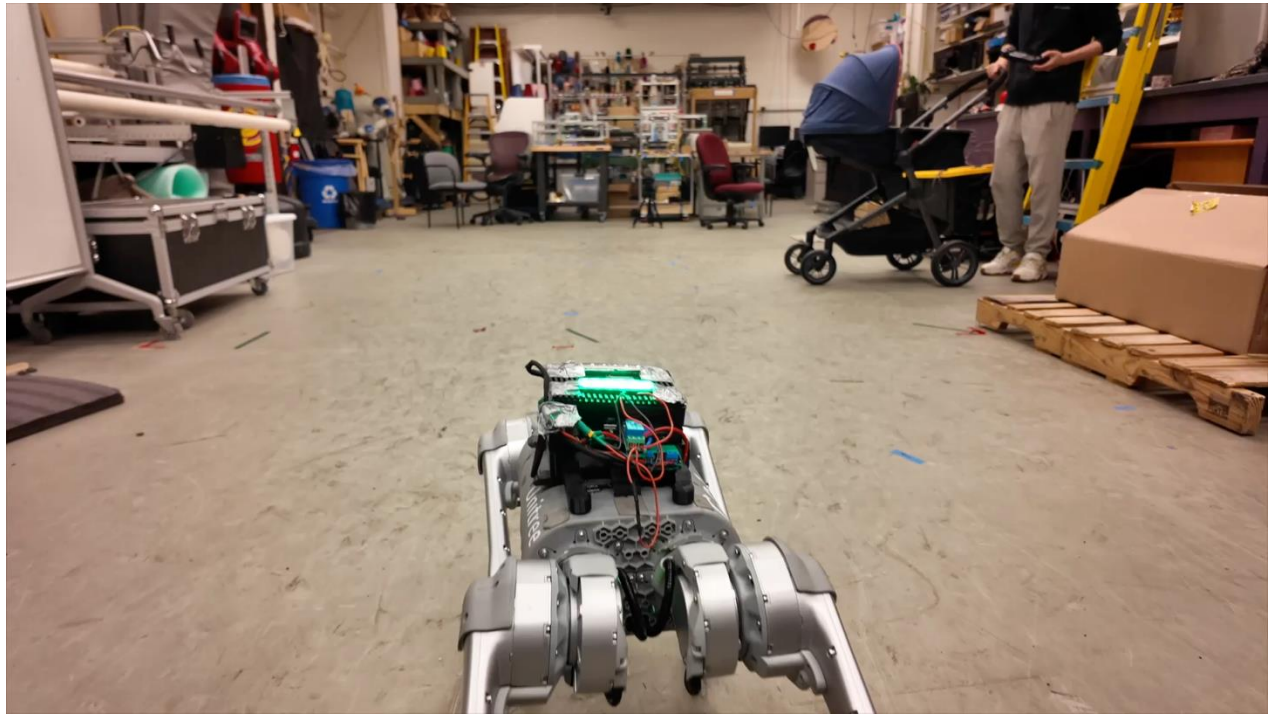


PPO in Robot Learning

- PPO becomes the go-to option in robot learning with continuous action space (especially in manipulation and locomotion)

Agile But Safe: Learning Collision-Free High-Speed Legged Locomotion

Tairan He[1†] Chong Zhang[2†] Wenli Xiao[1] Guanqi He[1] Changliu Liu[1] Guanya Shi[1]
[1]Carnegie Mellon University [2]ETH Zürich
[†]Equal Contributions https://agile-but-safe.github.io



RMA: Rapid Motor Adaptation for Legged Robots

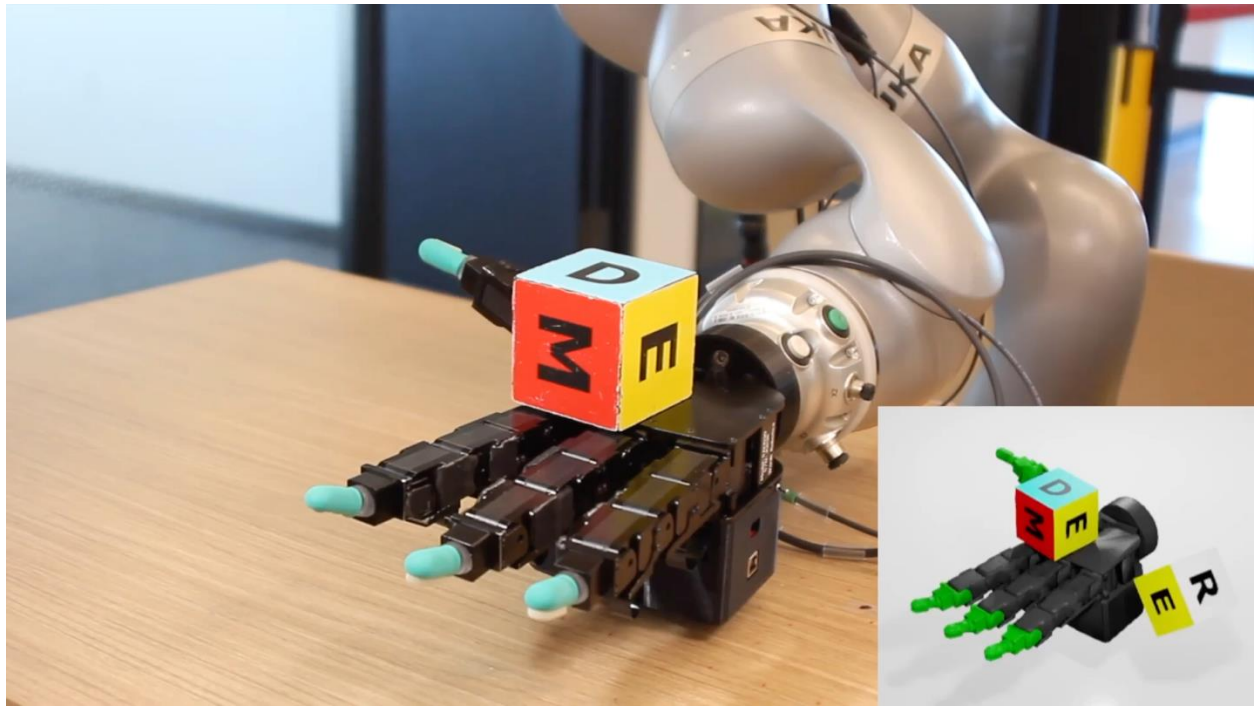
Ashish Kumar UC Berkeley
Zipeng Fu Carnegie Mellon University
Deepak Pathak Carnegie Mellon University
Jitendra Malik UC Berkeley, Facebook

PPO in Robot Learning

- PPO becomes the go-to option in robot learning with continuous action space (especially in manipulation and locomotion)

DeXtreme: Transfer of Agile In-Hand Manipulation from Simulation to Reality

Ankur Handa, Arthur Allshire, Viktor Makoviychuk, Aleksei Petrenko,
Ritvik Singh, Jingzhou Liu, Denys Makoviichuk, Karl Van Wyk, Alexander Zhurkevich,
Balakumar Sundaralingam, Yashraj Narang, Jean-Francois Lafleche, Dieter Fox, Gavriel State
NVIDIA.



Why PPO Works in Practice

- TRPO:

$$\theta' \leftarrow \arg\max_{\theta'} \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

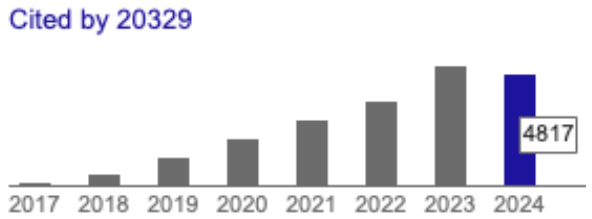
$$\text{such that } D_{\mathrm{KL}}(\pi_{\theta'}(\mathbf{a}_t|\mathbf{s}_t) \| \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)) \leq \epsilon$$

- PPO: Remove the constraint in TRPO, and change the red circle to ($\epsilon = 0.2$ works well):

$$L_t^{CLIP}(\theta') = \min(r_t(\theta')\gamma^t A^{\pi_\theta}(s_t, a_t), \mathrm{clip}(r_t(\theta'), 1-\epsilon, 1+\epsilon)\gamma^t A^{\pi_\theta}(s_t, a_t))$$

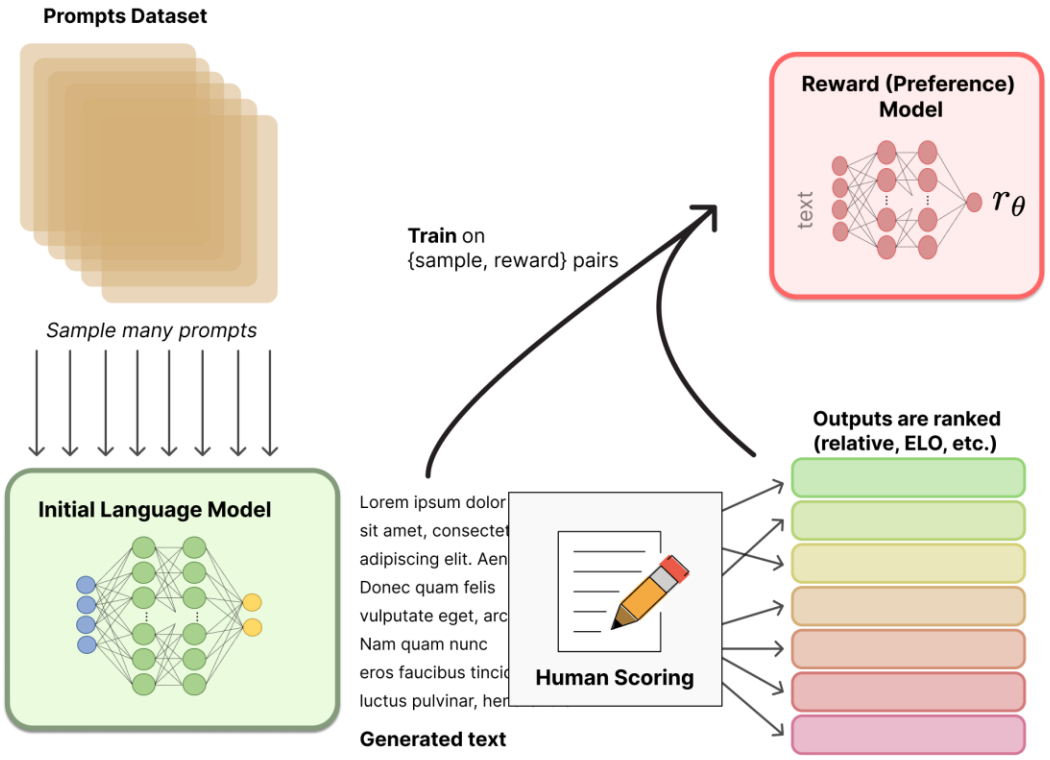
Proximal Policy Optimization Algorithms

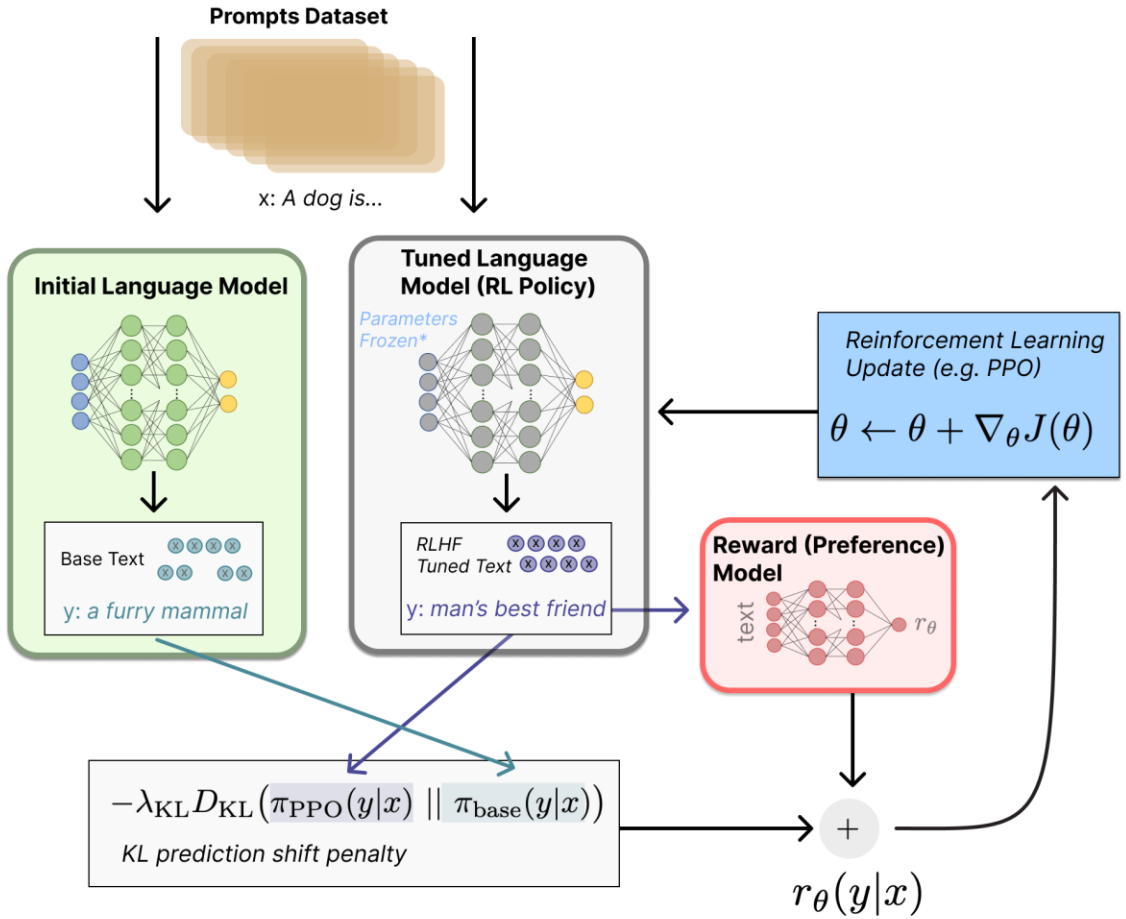
John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com



- PPO is very simple
- PPO is parallelizable
- PPO is robust: $L_t^{CLIP}(\theta')$ is a pessimistic estimation (lower bound) of the red circle
- Problem: no guarantee, $\epsilon = 0.2$ is a heuristic. Might be suboptimal for some problems

PPO in RLHF for LLM (GPTs)





https://huggingface.co/blog/rlhf

Summary

- ❑ Advanced **model-free** & **online** RL algorithms (algorithms people are using for robot learning) we have covered:
  - DQN and extensions (DDQN)
  - DDPG
  - SAC
  - NPG
  - TRPO
  - PPO
- ❑ Other algorithms:
  - A3C/A2C: for parallel training
  - D4PG (distributional DDPG)
  - MADDPG: multi-agent DDPG
  - …
- ❑ The best way to learn them is to implement them yourself!

Thank You!